# Final report - Projet intégrateur Group 6A

ABDOLMOHAMMADIAN Kian, ADAMAH Amele Olivia,
ANDRIANAMBININTSOA Rarivoson, BERTHELOT Sacha,
BOUBACHICHE Abderaouf, FAYEZ HANNAALLAH Karim Shady Monir,
HARZHANAU Anton, JIANG Tingting

May 13, 2025

# Contents

**1 Project description and technical details** **1**
1.1 Game presentation . . . 1
1.1.1 Context . . . 1
1.1.2 Game rules . . . 1
1.2 Project architecture and modules . . . 2
1.2.1 Program architecture and classes splitting . . . 2
1.2.2 Data management . . . 4
1.2.3 Server part and interactions with it . . . 4
1.2.4 Aesthetic aspect and HCI . . . 6
1.3 Final result . . . 8
1.4 Possible upgrades . . . 8

**2 Project organization and management tools used** **10**
2.1 Team splitting and main tasks . . . 10
2.2 Tools used for team organization . . . 11
2.3 Evolution of predictions . . . 12
2.3.1 Document modifications (Gantt) . . . 12
2.3.2 Solutions and answers to problems . . . 14

**3 Team members contribution** **14**

# 1 Project description and technical details

## 1.1 Game presentation

### 1.1.1 Context

This document details every key point of the project from team 6A. The objective was to create a game in a team from eight to eleven members to work on every technical aspect needed to develop a multiplayer online hosted game. For this project, we wanted to choose a game made for the whole family so everyone could enjoy it and spend great time with friends or even strangers, so we decided to take the basis of the famous "**Monopoly©**" board game with the mechanics of buying properties and paying other players in order to win.

The name of our game is "**Vacashon**" and consists of a trip around the world to become the new master of the touristic economy. To realize this project, we worked from February 5th to May 13th implementing the game mechanics, the server interactions and the designs to have a final result as close as a marketable game possible.

In the next pages, the key points of the game are detailed with their technical aspects to describe the main architecture of our project. It will be followed by the explanation of the organizational process within the team and how the work was split during the whole development. Finally, every team member described their work on the project adding details about their choices.

### 1.1.2 Game rules

"**Vacashon**" is a two to four players board game designed for the whole family heavily inspired by the "**Monopoly©**" board game. The main goal of the players is the be the last one who didn't go bankrupt or to have the highest value overall at the end of the timer.

Figure 1: Game board of "Vacashon"

To reach this objective, players must throw two dices to move around the board an buy properties hoping other players will land on them, owing them a rent. The properties are represented by cities of the world each split in different countries. If a player owns all the cities of a same country, they can buy buildings to increase their rent and make their opponents lose money quicker.

To help them traveling around the world and getting bonuses, special spaces are placed all around the board. Each has special effects or make players draw cards:

- Take-off: This space is the starting space of the game, when passing by this space again the player gets 200 of money.
- Go to Customs: Landing on this space sends the player to customs control. Like the 'Jail' in the Monopoly©, if players want to get out they can spend 50 of money on their next turn, roll the dices to attempt getting the same number on each dice, use a special 'Get out of Customs' card advantage, or wait three turns.
- Lucky Suitcase and Mystery Passport: These spaces make the player draw a card from the corresponding deck shuffled at the beginning of the game.
  The Lucky Suitcase cards can give or take money to players, make them move to different spaces or be a 'Get out of Customs' advantage card to keep for later.
  The Mystery Passport cards are only movements to different cities on the board, making the player move without having to throw the dices. When landing on the indicated space, regular actions can be made such as buying the city if free or paying the rent to the owner.

The game ends when only one player didn't go bankrupt or if the timer hits zero. In that case the value of the buildings, properties and money are added for each player to determine who won the game.

## 1.2 Project architecture and modules

To develop this project, we needed to focus on four necessary key points to make the game work, to manage the data of the players and the states of the game, to make the game accessible online and to give a real visual identity to it. Here are their description and the explanations about the different choices made to develop these parts:

### 1.2.1 Program architecture and classes splitting

When deciding about the software used to develop our game, we wanted something simple and easy to understand for everyone which granted multiple possibilities to be freely able to design our board, make the mechanics and add new elements easily without being lost in intersecting nodes everywhere. Since someone in the team already had basis with Godot, we decided to choose it knowing it wouldn't be long to learn or manipulate after the training phase.

To organize the program, each object used in the game has **its own class** to be entirely separated from the others and manipulated easily if we want to change different characteristics or attributes. The game mainly focuses on interacting with **the server** but has to keep data on the client used on the player's device to be able to store information or display them. Here is the organization of our program and how the different functions are treated:

## Core Entities

- User – represents an authenticated account containing login, password, and token. Interacts through the HTTP API.
- Lobby – a virtual room where users connect before the game starts. It stores information about the owner, participants, available tokens, and status ('is_active').
- Player – a game-specific entity created from a 'User' now the game begins. It contains money, position, property list, and in-game status (e.g., in jail).

- GameState – holds the current state of the game: list of players, game board, active player ID, card decks, and last dice rolls.
- GameManager – orchestrates game operations and routes incoming player actions to logic handlers.
- ConnectionManager – maintains all active WebSocket connections and tracks associated user data.
- GameHandler – the WebSocket event processor. It handles client messages, delegates them to the 'GameManager', and dispatches responses.

## Game Communication

- Players send actions via JSON messages over WebSocket: {"action": "roll_dice"}
- 'GameHandler' extracts 'player_id' from the socket and delegates the action to 'GameManager.process_action(...)'.
- 'GameManager' verifies turn ownership and invokes the appropriate handler from 'action_handlers'.
- Game state is updated and serialized using Pydantic, then dispatched:
  - Personal messages via 'send_personal_message()'.
  - Broadcasts via 'broadcast()' (e.g., turn changes, movements).

## Game Logic

Game logic is encapsulated in 'GameLogic' and includes:

- Dice rolling and validation;
- Player movement with bonus for passing "start";
- Cell activation logic ('cell.activate()');
- Property purchase/sale, house management;
- Event cards;
- Jail decisions.

The game board is loaded from a structured JSON file into a 'GameBoard' object, which contains strongly typed cells (streets, railways, events, utilities, corners). Actions triggered depend on the cell's type.

### 1.2.2 Data management

To be able to store player's name, login, password and the current states of the board as well as displaying the data in real time, we needed a Database in order to make our game work as fast and efficiently as possible.
When came the choice about the structure and the type of database we wanted to use, we made **two different prototypes**. In a first time we developed an API using FastAPI and SQLite, and in a second time we did an other prototype in PostgreSQL to compare the performance and capabilities of both DBMS to make the best technical choice.

This testing phase allowed us to evaluate the strengths and limitations of each solution. Ultimately, after discussions with the team, we chose PostgreSQL because it met our current needs: it is powerful, allows to create objects instead of unique fields and is safe to use. SQLAlchemy enabled us to define the models directly in the code while automatically generating the associated tables in the database. Additionally, Pydantic was used to define data validation schema. This allowed us to secure communication between the backend and the client by validating the structure and content of the data sent or received through the API.

Following discussions with the development team, it was decided to separate the types of data: On one hand, static configuration data, such as the types of board cells (PropertyCell, StreetCell, RailwaysCell, etc.), are managed in JSON files loaded at startup, as they define the initial game elements (the board). On the other hand, dynamic data, such as players and lobbies, are stored in the database since they are modified based on user actions.

### 1.2.3 Server part and interactions with it

## Architecture

The game is based on a client-server architecture. A centralized backend server validates and authorizes every client action, ensuring synchronization across platforms and supporting cross-platform compatibility.

#### University internal Version

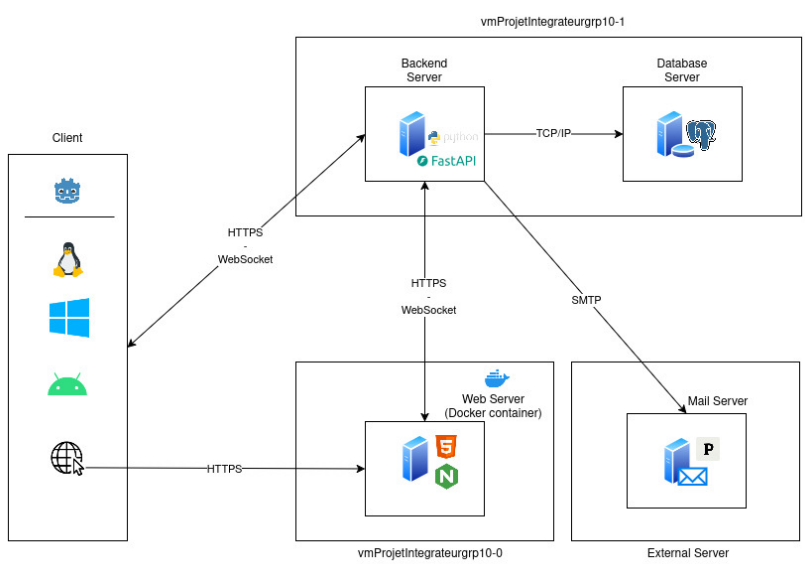


Figure 2: Technical Architecture - University version

**Public Version**

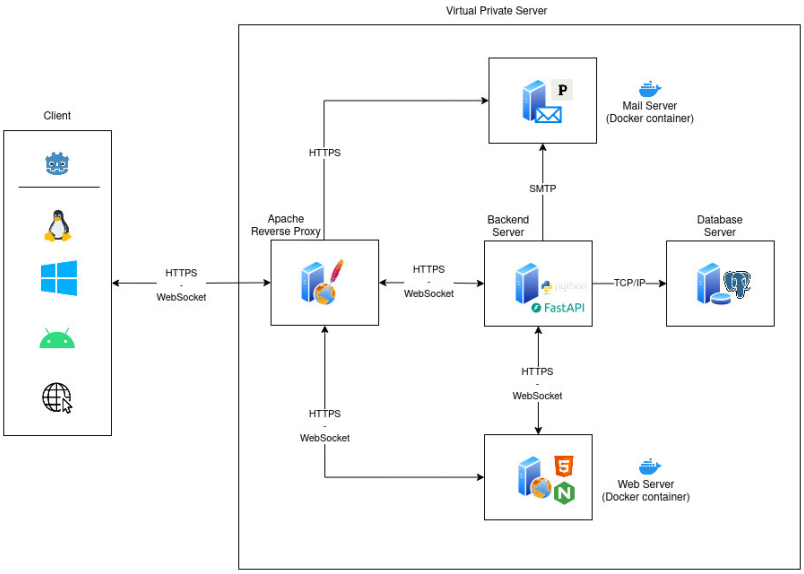


Figure 3: Technical Architecture - Public version

The table below summarizes the main technologies used in the development of this project, along with their versions and roles within the system.

| **Technology** | **Version** | **Purpose** |
|---|---|---|
| Python | 3.10.12 | Main programming language used to develop the game server |
| FastAPI | 0.115.11 | Python framework for building high-performance REST APIs and WebSocket endpoints |
| SQLAlchemy | 2.0.39 | ORM (Object Relational Mapping) tool for communicating with the PostgreSQL database |
| PostgreSQL | 14.17 | Relational database used to store game data, players and lobbies |
| Godot Engine | 4.4 | Game engine used to develop the multiplayer game client |
| Docker | 24.0.7 | Containerization of the backend, web server, and mail server for consistent deployment |
| Nginx | 1.27.5 | Web server used to serve the static web version of the game |
| Apache Server | 2.4.52 | Acts as a reverse proxy to route HTTPS traffic to backend and WebSocket services |
| Poste.io | 2.5.0 | Mail server to send emails from the custom domain |
| Git | 2.34.1 | Version control system used to manage and deploy code to the production server |
| HTML5/CSS3 | - | Technologies used to build a simple download page for the web version of the game |

Table 1: Vacashon Technology Stack

## Backend Server

The backend server was developed in Python using the FastAPI framework. This stack was selected after benchmark comparisons with other options such as **Godot's native networking server** and **Node.js**. Godot Server was not suited for complex backend logic, and lacked robust tooling for web development and API design. While Node.js is performant, Python with FastAPI was favored due to:

- FastAPI's**excellent support** for asynchronous programming, ideal for concurrent WebSocket connections.
- A clean and declarative syntax, speeding up backend development.
- Built-in OpenAPI documentation for easier debugging and API testing.
- Seamless integration with SQLAlchemy for database interaction.

## Networking

Two protocols were used for communication:

- **HTTP (for REST APIs)**: Used for user authentication, resource loading, and static data retrieval. REST was chosen for its simplicity, statelessness, and wide compatibility with HTTP clients.
- **WebSocket**: Used for real-time game communication (e.g. moves, game state updates). WebSocket allows full-duplex, low-latency communication essential for a better multiplayer online experience.

## Security

Security was implemented using an **SSL certificate** issued by Let's Encrypt, enabling secure HTTPS communication. All protected endpoints require user authentication to prevent unauthorized access to game data and personal accounts.

## Web Version of the Game

The game is also available as a web version, deployed via **Dockerized Nginx** on a remote server. This allowed fast and reliable deployment while supporting a wide range of users through the browser, without requiring installation. We weren't able to deploy it on the university's VMs because of the lack of resources they have to support our game and its components.

After making several tests we noticed multiple graphic aspects were missing and the entire authentication system didn't work which doesn't allow any player to connect to the game. That is why we decided to keep it hosted on a private server to operate our game services as they should in real conditions.

### 1.2.4 Aesthetic aspect and HCI

## Why we chose Figma

At the beginning of the project, our HCI team decided to adopt Figma as our primary design tool for both technical and collaborative reasons. Firstly, Figma enabled us to support a **three-stage prototype iteration process**, ranging from low-fidelity wireframes to high-fidelity interactive prototypes. Its features such as **Smart Animate** allowed us to simulate critical micro-interactions (e.g., dice rolling, event card pop-ups, etc.). Moreover, Figma offers a centralized design system management through master components, ensuring visual consistency across all screens, buttons, and cards.

In terms of collaboration, Figma was essential to **facilitate continuous interaction between designers, developers, and project manager** through role-based access control (editor, viewer, commenter), live design review sessions, and version management via Branching. For the development

handoff phase, we primarily relied on well-structured variants and modular components designed in Figma. These variants provided **clear, standardized, and easily implementable templates** for the development phase, enabling the development team to efficiently reproduce different interfaces and interaction states in Godot without relying on additional tools.

## Transition from Figma to Godot: Technical choices and architecture

Once the designs were finalized and validated, we initiated the technical migration to Godot, during which we made several structural choices:

- Modular Scene Architecture: Each menu (Home, Player Selection, Game Board) was implemented as an independent scene in Godot, enabling easier testing, iteration, and late-stage integration with the game mechanics team.
- Reusable Components and Responsiveness: We utilized control nodes (Control, VBoxContainer, HBoxContainer) to ensure adaptive behavior across different screen sizes (mobile, desktop).
- Pop-up Management via Dedicated Scenes: In order to realize the pop-up windows displayed when the player has an action to do in the different menus, we created separated scenes in Godot to make the interactions with the nodes and buttons as practical as possible for us to manipulate later when HCI team and Game Mechanics team started working together closely.
- Animations and Transitions: All animations and transitions were implemented directly inside Godot using GDScript, AnimationPlayer and node properties, based on the static design references from Figma, without the involvement of any external tools.

## Finalization of HCI Tasks and Integration with Game Mechanics

For the final HCI tasks, such as player information displays, action buttons, and contextual menus (event cards, purchase options, taxes, bankruptcy), we:

- Optimized menu code to ensure dynamic conditional transitions (e.g., display of password input when selecting private game mode).
- Implemented and adjusted all interactive pop-up menus, such as 'Chance' and 'Community Chest' cards, ensuring their graphical and functional consistency.
- Finalized the system for player movements and contextual interactions on the game board, working closely with the Gameplay team, particularly for special cells like airports.

Finally, the critical debugging and integration phase involved intensive collaboration with all teams to ensure that the HCI was not only functional but also well integrated with the game mechanics and compatible across platforms (Desktop, Mobile Web).

## Music and sound ambiance

In addition to the designs and graphics, we added music and sound effects to all the screens and menus to allow users to enjoy an audio ambiance while playing the game. In the settings, players can modify the volume of the music and SFX by using sliding bars. All preferences (music volume, SFX, vibrations, notifications, language, username) are saved and automatically reloaded thanks to a SettingsManager, ensuring data persistence between game sessions.

Background music is managed continuously and plays as soon as the game starts. Thanks to the connection between the sliders and the AudioServer, the players can adjust the volume at any time without interrupting the music. Sound effects (SFX) are played for every important action (button clicks, option changes, etc.), creating a lively and immersive atmosphere. The music only stops in specific cases (e.g., scene changes) and resumes automatically, ensuring a consistent audio experience.

## 1.3 Final result

After many months of work, we are proud to say we finished our game "Vacashon". Some mechanics we wanted to add to make the game even more unique are missing as said in part 1.4, but we are happy about the result and the work we are delivering today realizing that what we had in mind and modeled on Figma became reality in Godot.

To briefly recap what the game offers to the player in its final version, here is a list of the features that are available to the use and the possibilities the main game gives to the players:

- An authentication menu allowing to create an account or log in if an account is already created.
- A 'forgotten password' feature sending a code by e-mail to enter a new password if needed.
- Different menus on the welcoming screen to explain the rules of the game, give a feedback and adding a note to the user experience and also a settings menu allowing the players to change the volume of the music and sound effects.
- Possibility to create public or private rooms using a code from 2 to 4 players.
- In game, players can buy properties, buy buildings if they have all the cities of a same country and sell their buildings and properties if they are out of money.
- In game, event cards make players move to different spaces, lose or get money.
- In game, players can travel to the next airport when owning one and landing on it again. If the next one is owned by someone else they need to pay the rent. If it is free or owned by themselves, they just move there, ending their turn.
- In game, corner spaces work the same as "**Monopoly©**". If players are stuck in customs, they can pay, roll a double score with dices, use a 'Get out of customs' advantage card or wait 3 turns.
- Music and sound effects on the different screens of the game immerse the players into the ambiance of our game creating a pleasant and fun environment.

Figure 4: Screenshot of a game with four players.

## 1.4 Possible upgrades

During the entire development of the project, we had different ideas that would have been great to implement into the game to allow players to personalize their experience and never get bored when playing "Vacashon". Unfortunately, when we realized we were missing time to make these new mechanics possible we decided to cut them off the project and focus on the main game to have a fully functional product at the end. Here are the few new mechanics and features we wanted to add to our game if we had more time to do so:

- Changing username - During the development, we wanted to allow the players to change their username if they wanted to in the 'Settings menu' on the main menu. Being rushed at the end of the project we didn't have time to fully display and implement the corresponding field into the game. For this task it would have taken **5 hours**.

- Camera system - Originally we wrote in the specifications that we planned to make a camera system allowing players to follow the position of their token and to have a closer look at the board more easily. But seeing the time we had left to solve the multiple display and mechanics problems we unfortunately abandoned this objective. While being so focused on the last tasks, we didn't even think about how to implement it, so considering the time it would take to find a way, make tests and make it fully working it would probably take around **25 hours**.

- Tutorial - Since our game is heavily inspired by the "**Monopoly©**" game, we know there is some rules that can be difficult to understand at first sight. It is why we thought it would have been useful to implement a tutorial right after a player creates an account and launches the game. It would have been a predefined scenario to explain all the game mechanics of the game briefly explained by our main character himself to look like a little conversation with the player. Implementing this scenario considering it would be displayed only on first connection and be available to be played again if needed it would certainly take **20 hours** to make.

- New background themes - When we had the idea of making a vacation themed game, we knew that not everyone who would play the game have the same definition of what vacations truly are to them. That is why we initially wanted to allow players to choose the background of the screen when playing. In the current release, no choice is proposed to the players and only displays a sand beach. But in the Figma model we also had plans for a **snowy theme** and we had ideas for a **mountain theme** and many others could have been added to let the choice.
  Since it is mainly a matter of displaying a background and changing the background music and adding a field to allow players to choose the theme, the estimated time to implement one new theme would be **7 hours**.

Figure 5: Idea for snowy theme in lobby finding menu.

- New boards - Following the logic of the previous point when adding new backgrounds, we also thought it would be great to add new boards to our game since it is themed to world travel. The current board contains the countries where all the members of our team are from, but we also thought that if we had enough time to do it we could have created a real **world travel board** adding America and Oceania or even an **European trip board** with only pargetting a single continent. Many options were possible but considering the time it would take to make the designs and enlarge the database the estimated time to realize a single new board would be **20 hours**.

- Adding AI controlled bot players - In the case of a player waiting to play alone, we thought it would be a great idea to implemented **AI bots** not to worry about finding a lobby or wanting for other players to join theirs. When thinking about it we hesitated between making **basic AI** which would make random actions being limited with the available ones which could have occurred fun but incoherent situations. Or we also thought about making **advanced AI** which could take decisions depending on the amount of money they have, the number of buildings they bought, the properties they already own and other features. Making this would have taken a lot of time in addition to make buttons adding bots or letting the player choose the number of bots they want to play with, so for the basic version it would probably have taken **10 to 15 hours** and for the advanced model **30 to 40 hours**.

- New game modes - At the very beginning of the development, after deciding the theme of the game, some members suggested additional **game modes** that could have been added if we had enough time. One suggestion was playing as a big company owner and competing against other businessmen to buy different companies and even buy **company shares.** When buying shares, players would have taken a part of the rent due to the owner when someone lands on a property. Additional twists were also an idea for this game mode adding **fluctuating money rates** which would have made the game way more difficult and not for the whole family anymore. But considering the new game mechanics as well as a special board and look, it would have been a total redesign to implement this and would probably have taken **70 to 80 hours of work**.

# 2 Project organization and management tools used

To be able to get the project to the end and making all the technical aspects work together, the key to our progress was **organization**. To do that, team members were split into separate working teams to complete different tasks and interact with each other when we needed to connect features together.

In the next parts, we will explain our methodology and how our work was organized during the whole development process and how our expectations evolved with time comparing the first and final Gantt diagrams. We will also review the difficulties we had and how we managed the problems to be able to keep making progress without being locked for too long.

## 2.1 Team splitting and main tasks

Before making the different teams, we wanted to know what each member wanted to work on and in case they were not at the same skill level, if they wanted to be helped to progress and realize important tasks on their own. This is why after discussions this is the team splitting we ended up with and which stayed the same during the whole development:

- **Game development team**: Amele Olivia, Anton, Abderaouf
  This group was responsible of developing the game mechanics as well as how the player can play, which input they should use to play and how the game works by itself. They were joined after some time by the Human Computer Interface team to agree on the places of each buttons and what each should do and where it should display on the screen. They were also helped by the server implementation team to create the interactions between the game mechanics and the server hosting every action out of the client, letting the server managing the lobbies and every game actions.

- **Server implementation team**: Kian, Rarivoson
  This group worked on the interactions between the server and the game for the different clients. They also implemented a way to create a lobby and join it while making it visible on a connection menu and make the game work simultaneously on all connected devices at a time. In order to do that they firstly made a prototype on a local server hosted on Godot and then extracted it on university's VM servers.

  When wanting to make a WEB client, they noticed the VM's web resources weren't enough to make our game work so they uploaded it on a private server. In addition to the interactions, they also created the identification system to allow players to create an account and even receive a code by e-mail if they forgot their password to reset it.

- **Human Computer Interface team**: Karim, Tingting
  This group had to design the interface of each menu and the main game. They drew sketches to share their vision of the final look of the game and menus on Figma taking the basic design of the "**Monopoly©**" app then got helped by the project manager to create unique designs to give our game a real look getting further from the original inspiration. Then they implemented the validated visuals directly into the code to make the game look like their ideas on the previously designed model.

- **Project manager**: Sacha
  During the project, he took the responsibility of project manager by organizing the meetings, taking news of the progress from each team member each week and writing the weekly reports as well as the meeting reports. When the Human Computer Interface team needed help to make the final designs on the Figma model he designed the board and the main screens giving them the direction to take to finish the design.

  When there were delays, he tried to talk with the person who was in difficulty or to understand the problem to know which other team member could help them if he couldn't himself. In addition to his project manager role, he also did the communication for the game, created social medias account, made posts and designed a flier displayed in the university's halls.

## 2.2 Tools used for team organization

To organize our work and to follow the tasks that were done, in development or coming next and to be able to get help when needed, we used different tools and software to communicate with each other outside of the weekly meetings. Here are the different tools we used to structure our works and to be able to talk to each other when we needed to:

- Discord - When beginning the project, we initially wanted to keep a WhatsApp group following the group for the whole 3rd year of Computer Science License students. But after our tutor asked us to create a Discord server to be able to communicate with him easily, we organized it by splitting the rooms for documents, help, vocal meetings and giving every team separated workspaces for the different teams to be able to work together calmly.
  The link to our Discord server can be found in the **Appendix**.

- GIT - As we learned in **Génie Logiciel** course during the previous semester, we were conscious about the importance of GIT to manage our work and be able to simultaneously develop on our sides without creating conflicts and erasing someone's work. Using GIT allowed to us to release different key versions before the final one and to be able to develop from this released version and keep implementing new features and solving problems. Merging has sometimes been a problem with some members work being from older project versions than others but the conflict were solved after some upgrades of the old codes, allowing us to be with the last version we released.
  The link to our GIT repository can be found in the **Appendix**.

- Gantt Project - To realize the Gantt diagram, the project owner decided to use Gantt Project because learned to use it during previous studies. This tool is simple to use to add tasks, affect members to them, evaluate the time it would take to realize them and project the impact delays could have on the whole project. It was really useful to create our diagrams even with hundreds of tasks and allowed us to easily export them to constantly update them on the common repository.

- Trello - Alongside Gantt Project, we used the Trello cardboard website to illustrate and organize the tasks mentioned in the Gantt diagram in a clearer and easier way for everyone to understand them. This website allowed us to put the tasks in different categories corresponding to each working team and to easily set deadlines and affect persons to work on specific ones. When there were delays, it sent notifications to know who was late and which task needed to be modified which made us modify the Gantt diagram as well. It was our biggest help to keep track of the time we were taking on the most important tasks and to know who had finished the work they had to do.
  The link to our Trello cardboard can be found in the **Appendix**.

- Google Drive (Sheets, Docs, Forms) - To be able to calculate the number of hours each member did on the project for this report as well as writing the different weekly reports, we needed a tool to write the different tasks of everyone, the number of hours they worked and the difficulties they had. That is why we decided to use a Google Drive repository to gather all these information into one place as well as the Gantt diagram in a PDF file and other important files such as the functional specifications. To get everyone's feedback we used a Google Form to let every team member write their work and the time they took to realize them as well as their difficulties. And

we also used a Google Sheet table to report these information per date and calculate everyone's total working hours on the project at the end.
The link to our Google Drive repository can be found in the **Appendix**.

## 2.3 Evolution of predictions

At the very beginning of the project we decided a number of tasks that seemed coherent to us. During the whole programming process we discovered and defined new tasks that were important and that we didn't realize at first when talking about the concept and the way to make the game. It is why throughout the development we added and removed tasks we didn't think they would be possible to make with the time and work left. Here are the different **Gantt diagrams** from the specifications from February 14th and the final one from April 30th:

### 2.3.1 Document modifications (Gantt)

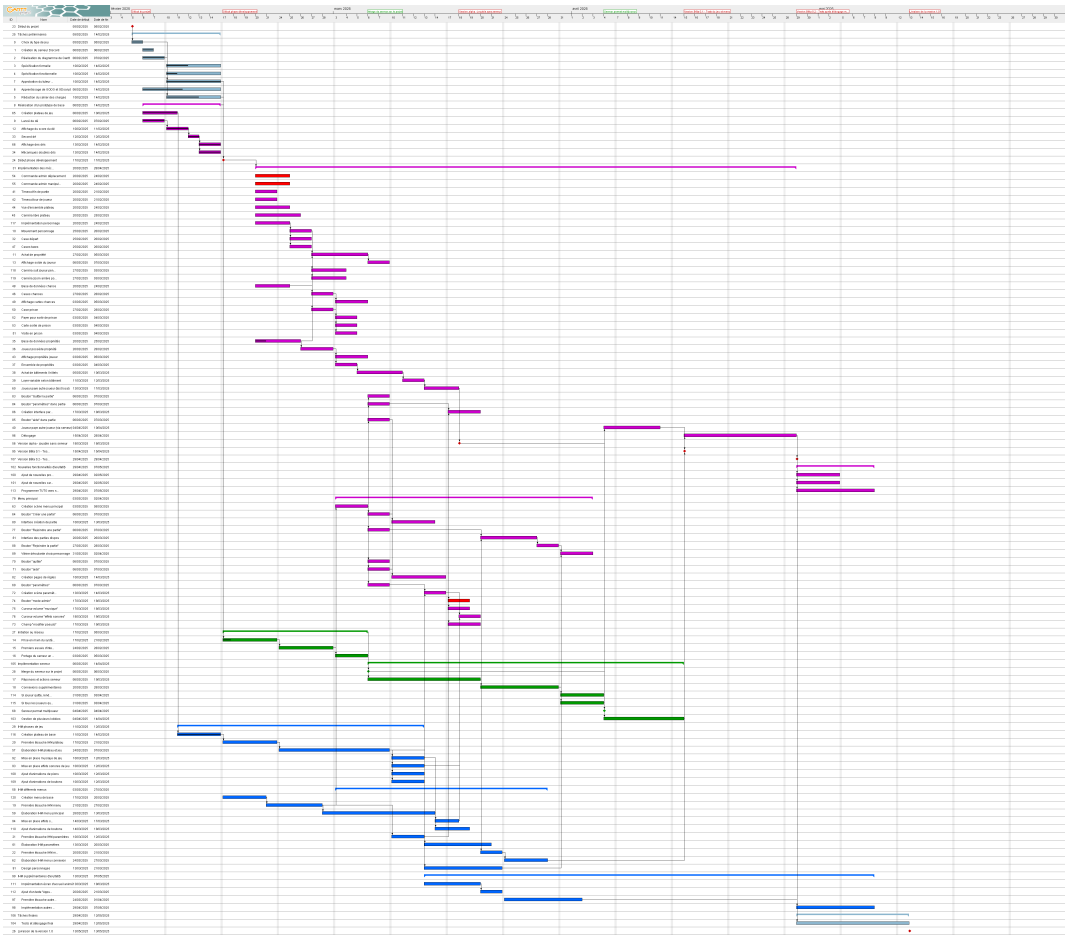

Figure 6: First Gantt diagram from February 14th.

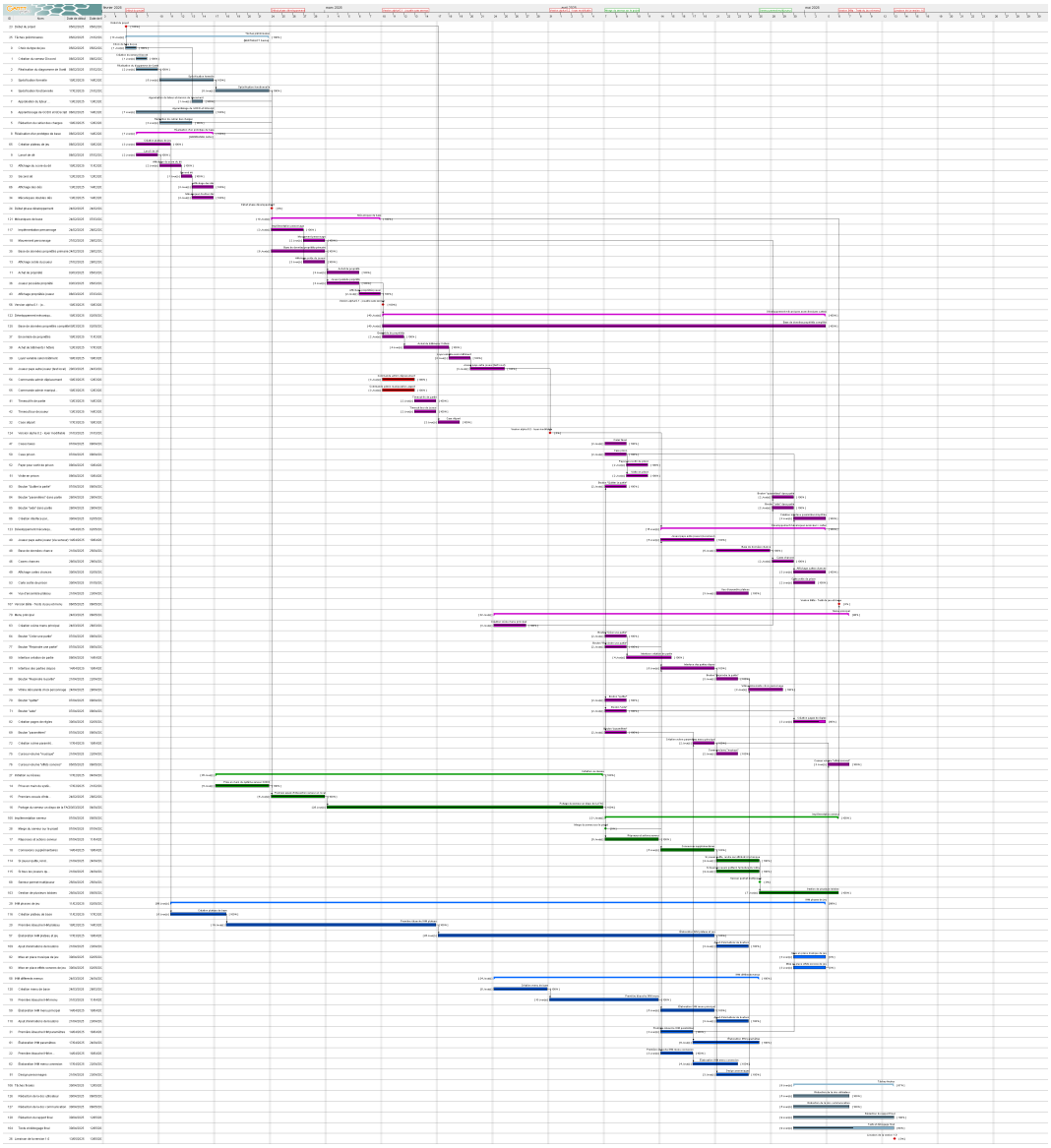

Figure 7: Final Gantt diagram from April 30th.

The biggest change between our expectations and the reality of the project has occurred during March. As we can see on the diagrams, most of the tasks planed for this month were delayed to April and the server and HCI tasks were lengthened. During this period we had our first exams and first projects to work on in the other courses which took us a lot of time and was difficult to handle at first.

Since the Figma model wasn't finished, we weren't able to start implementing the designs into the program not knowing the precise look we wanted to give to our game. We had the same kind of problem with the server part because at this point we were still working on a local hosted server without tests being made on the university's VMs. And since the next part of the game mechanics team was about programming the functions that would be managed and executed by the server the whole project was progressing very slowly, with everyone trying their best to match everyone's pace.

In addition to the delays, some tasks were canceled after noticing there was some essential tasks missing. For example, in the specifications, we precised we wanted to make a **following camera system** and **zoom mechanics** allowing players on mobile to follow their movements more easily and take a closer look at the board. But still having issues with the authentication system and some game

mechanics and displays, we preferred to cut this feature off the project and focus on the main work that was still to be done.

### 2.3.2 Solutions and answers to problems

The role of the project manager, in addition to writing the reports and splitting the work between the teams is to find solutions to technical and social problems. Fortunately everyone worked together and helped each other more or less during the whole project when needed. Although, our biggest enemy during this project was time. Our biggest worries were about our ability to finish features and to fix problems in time without delaying the whole project.

For example, one key feature that was important for us was to display the texts from the different properties on the board dynamically depending on the database's content. But after some delays and other projects the database was ready weeks later than it should have been done initially. To counter this problem, we worked based on a JSON file containing all the needed data to make a first test of dynamic display and when the database was ready we made it compatible with the structure used. For most of the technical parts we had temporary solutions to let people in charge of these big tasks to finish them and keep implementing other features based on the expected results of these ongoing tasks.

We also had a couple communication problems which led to misunderstanding and deleting works because the tasks they were answering to were already complete. For example Anton worked on making the filter to display only private and public rooms in the finding lobby screen and succeeded. One day later Tingting also worked on this task but wasn't present during the meeting where Anton did it, so she also implemented it and realized it was already done. Unfortunately we didn't keep her code but hopefully these problems didn't occur often and were about secondary tasks.

Fortunately we didn't have social problems though. Not everyone worked at the same pace or wasn't as efficient as others, but each member did their assigned work even if some had more work than others depending on their ability to handle multiple tasks at the same time or not. Although, the project manager had some issues contacting Abderaouf when reports weren't sent on time or when waiting for working recap. It was difficult to keep track of his work but in the end he had recurrent working recaps from him.

# 3 Team members contribution

Since each member of our team worked hardly to make our project possible and to fit our initial expectations as much as we could, everyone wrote their contribution to the development and final realization. Here are the written participation of each team member summarized in half pages paragraphs:

**ABDOLMOHAMMADIAN Kian**

I familiarized myself with the Godot game engine by developing a small server capable of handling multiplayer interactions with the high level API Multiplayer of Godot. Then, I worked on the implementation of a Python server handling Websocket to understand the technologies that we'll used later, I focused particularly on building REST API routes. A major part of my work involved user authentication features: I implemented a complete login system with a "remember me" option, account confirmation via email code using SMTP, and a password reset mechanism. This required working on both the Godot client side, to send requests, and the Python server side, to handle and respond to them properly.

I also implemented a system for sending a JWT token upon user login. This token is then used to authenticate requests made through WebSocket routes, ensuring secure real-time communication. Additionally, I designed and implemented the account confirmation and password reset pages, which had not been created yet. An initial version of this work was done on Nambi's branch, and later adapted and refactored for a newer branch. Finally, I carried out debugging tasks, including fixing a text display issue in the game rules section of the main menu.
**Total hours: 102 hours.**

**ADAMAH Amele Olivia (EXTRAIRE LA PARTIE BDD)**

I primarily worked within the development team on the database module and its integration with the API. I developed an initial API prototype using FastAPI and SQLite. I also created a second prototype using PostgreSQL.
To structure and interact with the database, I used SQLAlchemy, an Object-Relational Mapping (ORM) tool in Python that allows tables to be manipulated as Python objects.

Finally, in addition to my work on the backend, I also contributed to the client interface developed with Godot, by integrating the Settings and Quit buttons into the game interface. In summary, my contribution was essential to the entire data structuring and API design part, with a total of approximately **100 hours of work**.

**ANDRIANAMBININTSOA Rarivoson**

As a member of the server and network team, my main focus was to ensure that the multiplayer experience was seamless, secure, and accessible to all players regardless of their platform. I actively participated in the technical decision-making process, particularly in selecting the backend stack. After evaluating performance, ecosystem support, and development speed, we chose Python with FastAPI and SQLAlchemy, which offered a good balance between simplicity, speed, and robustness compared to other options like Node.js or Godot's native server.

On the infrastructure side, I handled the setup of the remote server, including the installation and configuration of core components such as Python, Docker, and PostgreSQL. I ensured that all backend services—REST API, WebSocket, and the database—were correctly deployed and interconnected. To make the server accessible across devices, I configured domain name settings and opened the necessary ports. The entire backend was secured using an SSL certificate issued by Let's Encrypt to allow secure HTTPS communication.

For deployment, we managed a Git-based workflows to update the production server. I also exported and deployed the web version of the game under our custom domain, `vacashon.online`, making the game easily accessible to the public. Additionally, I configured a dedicated mail server using Poste.io, enabling us to send emails from trusted addresses like `no-reply@vacashon.online` and `contact@vacashon.online`, which was crucial for user registration and support systems.

Beyond infrastructure, I contributed to game features by designing and implementing a scalable lobby system. This system allowed multiple games to run concurrently, with players able to create or join public or private lobbies. I also collaborated with the design team to bring dynamic behavior to several client-side screens, transforming static layouts into interactive interfaces.
**Total of worked hours during the project : ∼156h**

**BERTHELOT Sacha**

In this project I was the project manager. I took this role really seriously and wanted to make the working conditions as good as possible while keeping a constant working pace. At the beginning and the end of each weekly meeting I went to talk with each group member to know what they worked on during the week and if they had particular difficulties so we could talk about it and find solutions. At the end of the meeting I asked what they worked on during the session so I could write their work for the meeting report.

In addition to keeping track of the work of the team, I drew the Gantt diagram and made the Trello cardboard to split the tasks between the teams and follow the progress of everyone. I carefully updated them as often as I could during the project to know what tasks must be done in priority and which ones must be removed or delayed. To ask everyone's work out of the weekly meetings I also created the Discord server and the Google drive repository that were really useful to communicate, get help from others and fill the time sheet to calculate the number of hours spend on the project.

Outside of my role of project manager, I also helped the Human Computer Interaction team designing

the final look of the game on the Figma model to give them the direction on how to work next. Finally, I also did the communication around the whole project by creating the social medias, managing them, creating posts and making fliers visible from other students at university. To recap my working time on the whole project, I dedicated **135 hours** to Projet intégrateur in total.

**BOUBACHICHE Abderaouf**

I designed an intuitive interface where the user can adjust music and sound effects (SFX) using sliders, whose values are directly linked to Godot's AudioServer. This connection allows real-time volume changes, making the audio experience dynamic and changeable. I also integrated a text field that allows the user to change their username, accompanied by a confirmation button to ensure the change is explicitly applied.

To structure access to the features, I developed a clear and ergonomic settings menu, following a logical hierarchy of components. I included a popup window displaying the game rules, accessible via a button, and designed using a ScrollContainer to allow smooth reading even if the content exceeds the screen size. In the main game scene, I added three essential buttons: the settings, the rules of the game and quit to leave the current lobby. I also integrated a visible on-screen timer that displays the elapsed time in the session, allowing players to track how long they have been playing.

To make the interface more fluid and responsive, I added animations for hover and click interactions on the buttons. I used the create_tween() method directly in code, without creating a separate Tween scene. This work provides a feeling of immediate responsiveness and makes interactions more natural and engaging. These animations not only enhance the visual appeal but also strengthen user feedback, which is essential for good usability. **Hours worked on the project: 105h**.

**FAYEZ HANNAALLAH Karim Shady Monir**

**Design and Prototyping:** I created functional diagrams to visualize the game mechanics, designed the initial board on Figma, and developed various assets such as tokens, property cards, and the theme of the game. One of my key contributions was the idea to base the game's theme on eight countries, each representing a different nationality from our team, which added a unique and personal touch to the game. I also worked on modifying the game board graphic design and adjusting property cards, ensuring their size and text fit consistently.

**Collaboration and Coordination:** I worked closely with the HCI team, regularly meeting to clarify design ideas and resolve any discrepancies. We collaborated on the final design of the board, characters, and popups, ensuring they matched the game's theme. We also discussed and implemented the design for various in-game elements such as the "Chance" and "Community Chest" cards, ensuring they were integrated smoothly with the overall aesthetic.

**Implementation in Godot:** After creating and refining assets in Figma, I began implementing them into Godot. This included tasks such as setting up event cells, corner cells, and property card displays, along with modifying the background and popup designs. I also worked on coding features like player info display, adding buttons, and adjusting menus to fit the game's theme.

**Debugging and Problem-Solving:** Throughout the project, I contributed significantly to debugging to ensure better functionality and a smoother user experience. I helped identify and fix issues with the design implementation in Godot and reorganized asset files for better management. I also worked with the team to resolve coding problems such as taxes and bankruptcy mechanics, and solved issues related to card displays and hierarchy.

**Adjustments and Collaboration:** I worked closely with Anton to implement new mechanics, such as moving players to airports, and ensured that all design elements were well-integrated, properly displayed and functional within the game environment.
**Total hours worked: 135 hours.**

**HARZHANAU Anton**

As part of the development of the multiplayer online game "Vacashon", I worked on both the server and client sides, taking responsibility for the technical implementation of the game mechanics. On the server side, I contributed to the architecture design based on FastAPI and the implementation of real-time interaction via WebSocket, enabling data exchange between players. I participated in developing the lobby management system: creating and joining lobbies, selecting tokens, starting the game, as well as handling player connections and disconnections. The game state is transmitted to clients as serialized JSON objects, with their structure defined using Pydantic. I also implemented core game logic, including turn processing, activation of event cards, property purchasing and selling, balance management, and house construction.

On the client side, using Godot 4, I created a prototype user interface, including property cards, the lobby screen, and a visual representation of the game board. I implemented token movement for players, considering special tiles, and developed a system for displaying player information and actions. I also created interface prototypes for event cards along with their underlying logic. In addition, I set up WebSocket message handling on the client to ensure synchronized gameplay. The entire project was built with modularity and scalability in mind, allowing for future functionality expansion.
**Total hours: 180**

**JIANG Tingting**

In this project, I collaborated with Karim on the Human-Computer Interaction (HCI) team, where we were responsible for the design and implementation of the game's user interface and interaction logic. I led the UI prototyping and its full implementation in Godot, managing the entire process from initial concept to final integration.

The HCI module was one of the first components developed and served as a foundation for the overall user experience. Once the core gameplay mechanics were defined by the team, I began designing the user interface from scratch using Figma—learning the tool while applying it in a real project setting. Despite having no prior experience with Figma and no reference templates to follow, I successfully delivered a complete and functional prototype. This included carefully planned page layouts, structured screen compositions, visual design for player cards and interactive buttons, and the styling of the game board. I also created interfaces for token selection and player count options, applied consistent color schemes to ensure thematic cohesion, and drafted initial logo designs with the help of AI tools. Additionally, I designed interactive transitions and animation flows to enhance user engagement.

After finalizing the design, I moved on to the technical implementation in Godot. I structured each page using nodes and adapted the layout for responsiveness across different screen sizes. I developed animation effects and pop-up behaviors using GDScript, and implemented essential UI functionalities such as button actions, scene transitions, and conditional elements—such as displaying a password field for private game rooms. I also optimized the code to ensure that user interactions were consistent with the intended design and smooth in execution.

Throughout the development process, I worked closely with the back-end and game logic teams to ensure seamless integration of the HCI module into the overall system. While human-computer interaction may not directly involve core game algorithms, it plays a crucial role in shaping user engagement and perception. This project allowed me to quickly gain proficiency in new tools and deliver a polished, interactive, and user-friendly interface that significantly contributed to the success of the game.
**Total of hours spent on the project: 125 hours.**

### Conclusion

Projet intégrateur gave us the opportunity to work in realistic conditions even if we were guided during the whole development process by advices given by our tutor. We truly realized how difficult it could be to keep track of time and to manage delays if we don't stay focus at one point of the project. Fortunately, the overall ambiance was pleasant and we didn't have any social problem which allowed us to work with confidence at our own pace during these three months.

Our project isn't totally the one we expected to have initially, missing some functionalities and aesthetic features that could have made it even better looking but we are still proud of the work we delivered still having to follow other courses and projects. Everyone did what they could and worked seriously during each meeting, asked questions and for help when needed. The inner communication got better and better after each week when different teams had to work together to merge important parts of the project and made possible the game we have today.

We would like to thank our tutor Mr. Guillaume Décor for helping us staying on the track when we were making mistakes especially at the beginning of the project and for his reactivity when we needed details or help when we had doubts about technics and approaches to develop not wanting to get into a colossal work without being sure.
Also thank you to Mr. Stéphane Cateloin for managing the whole course and reacting quickly when we had problems with the university's VMs during the server development and web exportation.

Thank you again for giving us the opportunity to make us work in teams to prepare ourselves for the projects that await us in our future employment and will surely be helpful no matter the professional career we will have after the studies.

## Appendix

- Link to our Discord server
- Link to our GIT repository
- Link to our Trello cardboard
- Link to our Google Drive repository